Specification: lta_cation_template.py

2026-07-21

Contents

1. Purpose and stage boundary **2**

2. Dependencies and provenance **2**

3. Public data structures **2**
- 3.1 `ExpansionProvenance` . . . 2
- 3.2 `TemplateAtom` . . . 3
- 3.3 `TemplateVariant` . . . 3
- 3.4 `TemplatePhysicalSite` . . . 3
- 3.5 `TemplateGroupingParameters` . . . 3
- 3.6 `ParserProvenance` . . . 4
- 3.7 `LTATemplateModel` . . . 4

4. Public function **4**
- 4.1 `load_lta_template_model` . . . 4
  - Inputs . . . 5
  - Output . . . 5
  - Errors . . . 5

5. Expansion algorithm **5**

6. Physical-site grouping **5**

7. Deterministic ordering **6**

8. Site anchors and fingerprints **6**
- 8.1 Raw periodic centroid . . . 6
- 8.2 Framework-derived anchor . . . 6
- 8.3 Local fingerprint . . . 6

9. Invariants **6**

10. Edge cases **7**
- 10.1 Identity-only CIF . . . 7
- 10.2 Incomplete off-center variant library . . . 7
- 10.3 Insufficient oxygen environment . . . 7
- 10.4 Zero grouping cutoff . . . 7
- 10.5 `group_by_label=False` . . . 7

11. Tests required by MS0 **7**

12. Deferred work **7**

Current authority routing: supported Group-I/II species, exact integer populations, formal charges, and charge neutrality are owned by `doc/specs/v7_13_group_i_ii_cation_extension.md`; the Al24Si24O96 charge-equivalent dry-volume model and empirical endpoint provenance are owned by `doc/specs/v7_14_group_i_ii_volume_reference_extension.md`. This specification imports those semantics where chemistry or volume enters its surface.

1. Purpose and stage boundary

`lta_cation_template.py` implements the template half of **MS0**. It converts one CIF into an immutable, deterministic model containing:

- the fixed lattice;
- the expanded chemical framework;
- grouped physical 4R, 6R, and 8R cation sites;
- supplied positional variants;
- source-operation expansion provenance;
- deterministic physical-site anchors and local framework fingerprints;
- all parsing and grouping parameters required to reproduce the model.

The module does **not** generate a population, enumerate subgroups, optimize symmetry, rescale the cell, or serialize a final structure.

The existing random-population code remains backward compatible and is not redirected through this model in MS0. The maximum-symmetry implementation will consume `LTATemplateModel` directly.

2. Dependencies and provenance

Required packages:

```
Python >= 3.10
numpy
pymatgen
```

The module currently reuses the minimal CIF parser and low-level periodic utilities from `lta_cation_population_grouped.py`. The immutable model, provenance aggregation, stable identifiers, anchor construction, and row-order-independent ordering are package-specific.

Connected-component grouping uses a disjoint-set forest following:

R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," *Journal of the ACM* **22**, 215-225 (1975), DOI: 10.1145/321879.321884.

3. Public data structures

3.1 `ExpansionProvenance`

```python
ExpansionProvenance(
    source_atom_index: int,
    source_label: str,
    source_operation_index: int,
)
```

Identifies one raw CIF atom row and one declared source operation that generated an expanded coordinate.

A periodically deduplicated expanded coordinate may carry several provenance records.

3.2 TemplateAtom

```
TemplateAtom(
    atom_id: str,
    label: str,
    element: str,
    frac: tuple[float, float, float],
    occupancy: float,
    provenance: tuple[ExpansionProvenance, ...],
)
```

Represents one deterministic expanded framework atom or one ignored template atom.

Framework identifiers have the form:

```
F_000000
F_000001
...
```

Ignored-template identifiers use `I_`.

3.3 TemplateVariant

```
TemplateVariant(
    input_variant_id: int,
    frac: tuple[float, float, float],
    source_labels: tuple[str, ...],
    provenance: tuple[ExpansionProvenance, ...],
)
```

Represents one supplied positional variant after expansion by the source CIF operations and periodic deduplication.

`input_variant_id` is globally unique and deterministic within the loaded model.

3.4 TemplatePhysicalSite

```
TemplatePhysicalSite(
    site_id: str,
    ring_class: str,
    source_labels: tuple[str, ...],
    variants: tuple[TemplateVariant, ...],
    raw_centroid_frac: tuple[float, float, float],
    anchor_frac: tuple[float, float, float],
    anchor_method: str,
    local_fingerprint: tuple[tuple[str, float], ...],
)
```

A physical site is the discrete occupation object. Its positional variants are alternative coordinates for one ion and are not independent occupation sites.

Stable site identifiers use:

```
4R_0000
6R_0000
8R_0000
```

They do not encode incidental CIF labels.

3.5 TemplateGroupingParameters

Stores the complete grouping contract:

```
TemplateGroupingParameters(
    site_map: str,
    template_elements: tuple[str, ...],
    dedup_digits: int,
    variant_group_cutoffs: tuple[tuple[str, float], ...],
    group_by_label: bool,
    anchor_max_oxygen_distance_ang: float,
    fingerprint_size: int,
)
```

3.6 `ParserProvenance`

Records:

- parser name;
- absolute input path;
- source atom count;
- source operation count;
- expanded framework count;
- expanded template-variant count.

3.7 `LTATemplateModel`

```
LTATemplateModel(
    source_path: str,
    lattice: Lattice,
    framework_atoms: tuple[TemplateAtom, ...],
    physical_sites: tuple[TemplatePhysicalSite, ...],
    ignored_template_atoms: tuple[TemplateAtom, ...],
    source_symmetry_operations: tuple[SymmOp, ...],
    grouping_parameters: TemplateGroupingParameters,
    parser_provenance: ParserProvenance,
)
```

Convenience views:

```
model.physical_sites_by_ring
model.input_variants
model.site_ids
model.framework_ids
model.source_expansion_site_lookup()
```

The object is semantically immutable. Coordinate fields are stored as tuples rather than writable NumPy arrays.

4. Public function

4.1 `load_lta_template_model`

```
load_lta_template_model(
    input_cif,
    *,
    site_map="NA1:6,NA2:8,NA3:4",
    template_elements="Na",
    dedup_digits=8,
    variant_group_cutoffs=None,
    group_by_label=True,
    anchor_max_oxygen_distance_ang=3.75,
```

```
    fingerprint_size=24,
) -> LTATemplateModel
```

Inputs

`input_cif` Path to the source CIF.
`site_map` Maps template labels or label prefixes to ring families.
`template_elements` Comma-separated elements treated as cation-template rows rather than framework atoms.
`dedup_digits` Decimal precision used for deterministic periodic coordinate keys. Must be at least 4.
`variant_group_cutoffs` Per-family minimum-image grouping radii in angstroms. Missing entries default to 2.5 A.
`group_by_label` When true, only variants sharing the same source label can belong to one connected component.
`anchor_max_oxygen_distance_ang` Maximum distance from the raw variant centroid to the farthest oxygen used in the framework-derived ring-center estimate.
`fingerprint_size` Number of nearest species-labeled framework distances retained in the local fingerprint.

Output

One deterministic `LTATemplateModel`.

Errors

The function raises `ValueError` for invalid user parameters and propagates CIF parsing failures. It raises `TemplateModelError` when provenance ownership or periodic model construction is internally inconsistent.

5. Expansion algorithm

For a source atom coordinate $\mathbf{x}_a$ and declared operation

$$g_i(\mathbf{x}) = R_i\mathbf{x} + \mathbf{t}_i,$$

the expanded coordinate is

$$\mathbf{x}_{a,i} = g_i(\mathbf{x}_a) \pmod{\mathbb{Z}^3}.$$

Expanded rows are deduplicated by

$$(\text{element}, \text{label}, \text{round}(\mathbf{x}_{a,i})).$$

All generating pairs (a, i) are retained as provenance. Deduplication therefore removes duplicate coordinates without discarding the operation history required by later physical-site mapping.

Framework and template rows are expanded separately.

6. Physical-site grouping

For one ring family, create a graph whose vertices are expanded template coordinates. An edge joins variants i and j when

$$d_{\text{MIC}}(i, j) \le r_{\text{group}},$$

and, when `group_by_label=True`, their source labels match.

Connected components are computed with a disjoint-set forest. Each component becomes one `TemplatePhysicalSite`.

The physical capacity of a family is the number of components, not the number of coordinate variants.

7. Deterministic ordering

The model must be independent of raw atom-row order.

Framework atoms are sorted by:

1. chemical element;
2. wrapped fractional coordinate key;
3. source label.

Variants within a physical site are sorted by coordinate and label.

Physical sites are sorted by:

1. ring-family order $4, 6, 8$;
2. deterministic anchor coordinate;
3. sorted variant coordinates;
4. source-label set.

IDs are assigned only after sorting.

Source atom indices remain provenance only. They do not determine model ordering.

8. Site anchors and fingerprints

8.1 Raw periodic centroid

Variants are unwrapped relative to a reference variant using minimum-image fractional displacements, averaged, and wrapped back into the cell.

8.2 Framework-derived anchor

For an n-ring family, the loader selects the n nearest framework oxygen atoms to the raw centroid. When the farthest selected oxygen lies within `anchor_max_oxygen_distance_ang`, the anchor is their periodic centroid.

This produces the methods:

`nearest_4_oxygen_centroid`
`nearest_6_oxygen_centroid`
`nearest_8_oxygen_centroid`

When a plausible oxygen shell is unavailable, the model uses:

`variant_centroid_fallback`

The raw variant centroid is therefore a deterministic fallback, not the sole general definition of a physical site.

8.3 Local fingerprint

The local fingerprint is the sorted list of the nearest species-labeled framework distances:

$$F(s) = [(Z_1, d_1), (Z_2, d_2), ...].$$

It is periodic, row-order independent, and later validates physical-site images under symmetry.

9. Invariants

A valid model satisfies:

1. every framework ID is unique;
2. every physical-site ID is unique;

3. every input-variant ID is unique;
4. every variant belongs to exactly one physical site;
5. every provenance pair belongs to at most one physical site;
6. ring classes are restricted to `4`, `6`, and `8`;
7. all fractional coordinates are wrapped into $[0, 1)$;
8. all ordering is deterministic under raw CIF atom-row permutations.

10. Edge cases

10.1 Identity-only CIF

The model remains valid. A later `auto` symmetry backend may detect additional operations from a fully expanded P1 framework.

10.2 Incomplete off-center variant library

The template model records only supplied/expanded variants. It does not reject incomplete fibers. The symmetry module decides whether to complete the library or retain it strictly.

10.3 Insufficient oxygen environment

The anchor falls back to the raw periodic variant centroid and records that method explicitly.

10.4 Zero grouping cutoff

Every expanded coordinate remains a separate physical site, subject to exact coordinate deduplication performed during expansion.

10.5 `group_by_label=False`

Variants from different source labels may form one physical site. `source_labels` stores the complete label set.

11. Tests required by MS0

Focused tests must verify:

- deterministic framework/site/variant ordering after source atom-row reordering;
- stable IDs;
- one-to-one provenance ownership;
- correct ring-family grouping;
- anchor fallback behavior;
- compatibility with the existing synthetic C4 fixtures.

12. Deferred work

The following remain outside this module:

- population constraints;
- subgroup enumeration;
- orbit coloring;
- exact variant realization under a subgroup;
- stabilizer optimization;
- final structure serialization;
- symmetry-preserving density repair.